SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Assessment Tool Weightage: 10%

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 30**
Question Count: 5

Code of Conduct

I **K.MAHESH BABU(192525122)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: **K.MAHESH BABU(192525122)**

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A recommendation engine processes user clicks, search queries, and purchases from multiple devices. Reverse engineer the probable data types, ingestion flow, and analytics stages.
2. A ride-sharing company displays live traffic heatmaps and demand forecasts. Reverse engineer the likely distributed storage and processing backbone behind the system.
3. A fraud detection dashboard flags suspicious card transactions in near real time. Reverse engineer the probable big data security, streaming, and alerting mechanisms used.
4. A social media platform stores videos, text comments, hashtags, and user engagement metrics. Reverse engineer how structured and unstructured data are probably managed together.
5. A manufacturing IoT platform collects machine logs and maintenance events from many plants. Reverse engineer the likely integration tools, storage strategy, and analytics path.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of	Good understanding	Partial understanding	Poor understanding

		underlying big data architecture			
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1. Recommendation Engine

Question

A recommendation engine processes user clicks, search queries, and purchases from multiple devices. Reverse engineer the probable data types, ingestion flow, and analytics stages.

Answer

A recommendation engine collects information about user activities from websites, mobile applications, and other devices. The purpose is to understand user interests and recommend suitable products, videos, services, or content.

Probable Data Types

The system is likely to handle the following types of data:

1. Structured Data

- User ID
- Product ID
- Purchase amount
- Purchase date
- Device ID
- Product category

2. Semi-Structured Data

- Web logs
- Application logs
- JSON event records
- Search-event information

3. Unstructured Data

- Product descriptions
- Customer reviews
- Images
- Other text-based content

Probable Ingestion Flow

The likely data flow is:

User Devices → Event Collection → Data Ingestion → Distributed Storage → Data Processing → Recommendation Analytics → Recommended Content

When a user clicks a product, searches for an item, or purchases something, an event is generated. The event is collected and sent to the Big Data ingestion layer.

Data from multiple devices must be combined using a common user identifier or another suitable identity mechanism.

Data Storage

The system would require scalable storage because data is continuously generated by many users.

Historical user activity can be stored in distributed storage, while frequently accessed information can be maintained in faster storage systems.

Analytics Stages

The probable analytics process is:

Data Collection → Cleaning → Integration → Feature Extraction → User Behavior Analysis → Recommendation Model → Recommendation Output

The system may identify:

- Frequently viewed products
- Frequently searched categories
- Previous purchases
- Similar customer behavior
- Recently viewed items

These features can be used to generate personalized recommendations.

Design Decisions

The system likely requires:

- Scalable storage
- Real-time or near-real-time event ingestion
- Batch processing for historical data
- Data cleaning and integration
- User-profile creation
- Recommendation analytics

Conclusion

The recommendation engine is likely built around continuous event ingestion, distributed storage, and analytics that convert clicks, searches, and purchases into user behavior patterns. The architecture must support multiple devices and large-scale data generation.

2. Ride-Sharing Company – Live Traffic Heatmaps and Demand Forecasts

Question

A ride-sharing company displays live traffic heatmaps and demand forecasts. Reverse engineer the likely distributed storage and processing backbone behind the system.

Answer

A ride-sharing company continuously receives information from drivers, passengers, GPS devices, mobile applications, and traffic systems. Since location information changes continuously, the system requires a scalable architecture capable of handling high-velocity data.

Probable Data Sources

The system may receive:

- GPS locations
- Vehicle movement
- Ride requests
- Completed rides
- Driver availability
- Passenger locations
- Traffic conditions
- Historical ride information

Probable Architecture

The likely architecture is:

Drivers/Passengers → Real-Time Data Ingestion → Distributed Storage → Stream Processing → Analytics → Heatmap/Demand Forecast

Distributed Storage

A distributed storage system is likely used because millions of location and ride events can be generated. The data can be distributed across multiple storage nodes. This provides:

- High storage capacity
- Scalability
- Fault tolerance
- Parallel data access

Historical ride data can also be retained for demand forecasting.

Real-Time Processing

Live traffic heatmaps require real-time or near-real-time processing.
The system can continuously process GPS and traffic events to determine:

- Areas with heavy traffic
- Average vehicle speed
- Number of available drivers
- Number of ride requests
- Areas with high demand

Demand Forecasting

Historical data can be analyzed to identify demand patterns.
For example, the system may identify:

- Morning peak hours
- Evening peak hours
- Weekend demand
- Demand near airports
- Demand near shopping areas
- Demand during events

Probable Processing Model

The system can use both:

Stream Processing: For live traffic and current demand.

Batch Processing: For historical data analysis and demand forecasting.

Heatmap Generation

The city can be divided into geographical regions. Each region can be assigned values representing traffic density or ride demand.

The processed information is then displayed as a live heatmap.

Design Decisions

The system requires:

- Distributed storage
- High-speed data ingestion
- Stream processing
- Historical data storage
- Batch analytics
- Geographic analysis
- Scalable infrastructure

Conclusion

The ride-sharing platform most likely uses a distributed Big Data architecture combining real-time stream processing with historical batch analytics. This allows the company to display live traffic conditions while also forecasting future ride demand.

3. Fraud Detection Dashboard

Question

A fraud detection dashboard flags suspicious card transactions in near real time. Reverse engineer the probable Big Data security, streaming, and alerting mechanisms used.

Answer

A fraud detection system analyzes card transactions continuously to identify unusual or suspicious activities. Because financial transactions are generated at high velocity, the system requires real-time or near-real-time processing.

Data Sources

The system may collect:

- Transaction ID
- Card/account information
- Transaction amount
- Time and date
- Merchant information
- Location
- Device information
- Previous transaction history

Probable Architecture

The likely flow is:

Card Transaction → Secure Data Ingestion → Stream Processing → Fraud Analysis → Risk Score → Alert → Dashboard

Streaming Mechanism

Transactions are continuously sent to the streaming layer.

The streaming system allows each transaction to be processed shortly after it occurs.
This is important because waiting for batch processing could delay fraud detection.

Fraud Analytics

The system can compare current transactions with historical behavior.
Possible suspicious patterns include:

- Unusually large transaction
- Multiple transactions within a very short period
- Transactions from unusual locations
- Sudden change in spending behavior
- Multiple failed transactions

The system can assign a risk score to each transaction.

Security Mechanisms

Because financial transaction data is sensitive, the system requires strong security.
Important controls include:
Authentication: Ensures only authorized users and applications can access the system.
Authorization: Controls what each user can view or modify.
Encryption: Protects transaction information during transmission and storage.
Access Control: Limits access to sensitive financial information.
Audit Logging: Records system and user activities.

Alerting Mechanism

If a transaction exceeds a defined risk threshold, the system can generate an alert.
Example:
Transaction → Risk Analysis → High Risk → Fraud Alert → Dashboard/Investigation Team
The dashboard can display:

- Transaction details
- Risk level
- Reason for alert
- Transaction location
- Time
- Customer information permitted by access policy

Design Decisions

The probable design uses:

- Secure streaming ingestion
- Real-time analytics
- Historical transaction data
- Risk scoring
- Access control
- Encryption
- Audit logs
- Real-time alerts

Conclusion

The fraud detection dashboard is likely based on a streaming Big Data architecture. Real-time transaction processing allows suspicious activities to be identified quickly, while security mechanisms protect sensitive financial information.

4. Social Media Platform

Question

A social media platform stores videos, text comments, hashtags, and user engagement metrics. Reverse engineer how structured and unstructured data are probably managed together.

Answer

A social media platform handles different forms of data simultaneously. Videos and other media files are generally unstructured, while engagement metrics and user information may be structured or semi-structured. Therefore, the platform needs a Big Data architecture capable of managing different data types.

Data Classification

Structured Data

Examples include:

- User ID
- Post ID
- Number of likes
- Number of shares
- Number of followers
- Timestamp
- Engagement count

These can be stored in structured databases or analytical data stores.

Semi-Structured Data

Examples include:

- JSON user events
- Application logs
- Metadata
- Hashtag information

Unstructured Data

Examples include:

- Videos
- Images
- Audio
- Text comments

Large media files require high-capacity storage.

Probable Architecture

Users → Data Collection → Data Integration → Appropriate Storage → Processing → Analytics → Social Media Services

Storage Strategy

Different storage technologies can be used according to the data type.

Structured data: User profiles and engagement metrics can be stored in databases.

Semi-structured data: JSON and event logs can be stored in flexible document or distributed storage systems.

Unstructured data: Videos and images can be stored in large-scale object or distributed file storage.

Data Integration

An integration layer connects information from different sources.

For example:

Video Upload + User ID + Hashtags + Likes + Comments → Integrated Dataset

This allows the platform to understand the complete context of a post.

Analytics

The platform can analyze:

- Most popular videos
- Trending hashtags
- User engagement
- Popular content categories
- User preferences
- Content performance

Example

Suppose a video receives:

- 100,000 views
- 20,000 likes
- 5,000 comments
- 3,000 shares

The platform can combine these metrics with the video metadata and hashtags to determine whether the content is trending.

Scalability
Because social media platforms may have millions or billions of users, the architecture must be scalable. Distributed storage and parallel processing allow large datasets to be handled across multiple machines.
Conclusion
The platform should not force every data type into one storage format. Structured data can be stored in appropriate databases, while videos and other unstructured content can use high-capacity distributed or object storage. An integration layer connects these datasets so that analytics can operate across the complete user experience.

5. Manufacturing IoT Platform

Question
A manufacturing IoT platform collects machine logs and maintenance events from many plants. Reverse engineer the likely integration tools, storage strategy, and analytics path.

Answer
A manufacturing IoT platform receives data from machines, sensors, production systems, and maintenance applications across multiple plants. The data is generated continuously and must be integrated to monitor equipment and identify maintenance requirements.
Probable Data Sources
The system may collect:

- Machine sensor readings
- Temperature
- Vibration
- Pressure
- Machine operating hours
- Machine logs
- Maintenance records
- Failure events
- Production information

Probable Architecture
The likely architecture is:
Machines/Sensors → IoT Gateway → Data Integration/Ingestion → Distributed Storage → Data Processing → Analytics → Maintenance Decision
Integration Tools
Integration tools are required because different manufacturing plants may use different machines, systems, and data formats.
They can perform:

- Data collection
- Data transformation
- Data validation
- Data filtering
- Data integration
- Data transfer

For example, sensor readings from multiple plants can be converted into a common format before being stored and analyzed.
Storage Strategy
A distributed storage system is appropriate because large amounts of sensor and machine data are generated continuously.
The platform can store:
Recent Data: Used for real-time monitoring.
Historical Data: Used for trend analysis and predictive maintenance.
Maintenance Records: Used to understand previous failures and repairs.
Analytics Path

The likely analytics flow is:

Raw Sensor Data → Cleaning → Integration → Feature Extraction → Analysis → Prediction → Maintenance Action

Predictive Maintenance

Analytics can identify patterns that occur before machine failure.

For example, if vibration and temperature gradually increase before previous failures, the system can identify similar patterns in current sensor data.

The system can then generate a maintenance recommendation before the machine fails.

Batch and Real-Time Analytics

Real-Time Analytics can identify abnormal machine conditions immediately.

Batch Analytics can analyze historical machine data to identify long-term trends and failure patterns.

Benefits

The solution can help manufacturers:

- Reduce unexpected machine failures
- Improve equipment availability
- Reduce maintenance costs
- Increase production efficiency
- Improve machine life
- Schedule maintenance more effectively

Design Decisions

The platform should provide:

- Scalable data ingestion
- Integration across multiple plants
- Distributed storage
- Real-time processing
- Historical analytics
- Predictive maintenance
- Data-quality management
- Secure access

Conclusion

A manufacturing IoT platform is likely built using an integration and ingestion layer connected to distributed storage and analytics systems. Real-time processing supports immediate machine monitoring, while historical analytics supports predictive maintenance and long-term optimization.